04 / 26 — Phase 4 Assessment: React Architecture

> **Scope.** Lessons 01 through 27 of Phase 4 (note that 26 is this file and 27 — Styling architecture is part of the assessed material), with prerequisite knowledge from Phases 0–3 assumed. **Format.** Five parts: (A) conceptual explanation, (B) code and structure prediction, (C) debugging challenges, (D) implementation exercises, (E) architecture and reasoning. A detailed answer key, scoring rubric, remediation map and promotion criteria follow. **Time.** 5–6 hours. Closed notes for Parts A–C and E. Part D may be done in an editor with TypeScript and React type definitions available (no AI assistance, no copying from lessons). **Rules.** Write answers *before* reading the key. For every prediction, write the result and one to three sentences of reasoning. For every debugging challenge, name the symptom's root cause, the fix, and how you would verify the fix. For every architecture question, state your **criteria**, your **decision**, the **trade-offs you accept**, and at least one **condition under which you would decide differently**. ⚠ Version baseline: React 19.x (see ROADMAP §0). Items touching volatile tools (ESLint plugin option names, bundler options, CSS-in-JS library status, Feature-Sliced Design details, RSC bundler integration) are assessed for the *concept* and the habit of verifying against current documentation, never for memorised option names.

How the assessment is scored (summary)

Category	Parts	Points
Explanation	A (12 questions × 3)	36
Reasoning	B (8 × 3) + E (4 × 4)	40
Debugging	C (5 × 6)	30
Implementation	D (3 × 10)	30
Total		**136**

Pass: **≥ 80% overall (≥ 109 / 136)** *and* **≥ 70% in each category** (Explanation ≥ 26, Reasoning ≥ 28, Debugging ≥ 21, Implementation ≥ 21). Details in Scoring and Promotion.

Part A — Conceptual explanation (36 points)

Answer each in 4–8 sentences (or a compact list). Each question is worth 3 points: one point per key idea in the answer key. Causal links ("because…") matter more than vocabulary.

A1. (Lessons 01, 02) What distinguishes an *architectural* decision from a design or implementation decision in a React codebase? Explain coupling and cohesion in terms of *change amplification*, and say why "separation of concerns" can make a codebase worse when overdone.

A2. (Lessons 03, 04) Explain a component's responsibility as a *contract* (what it owns: state, effects, data, layout). Why was the presentational/container split popular, why is the "smart/dumb" label discouraged today, and what has largely replaced the wrapper-component form of the container?

A3. (Lessons 05, 11) State the Dependency Inversion Principle and explain who *owns* the abstraction. Name three real React mechanisms for injecting a dependency and one reason a service locator hidden in a hook is worse than explicit injection. What is a composition root?

A4. (Lessons 06, 07, 09) Compare organising by technical layer with organising by feature using the *axis of change* argument. In Feature-Sliced Design, state the import rule between layers and between slices of the same layer, and say what a public API (`index.ts`) is for and what its cost can be.

A5. (Lessons 08, 10, 12, 13) Where should domain logic live in a React application, and why should it not import React? Distinguish domain, application and presentation logic, define "port" and "adapter", and give one situation in which hexagonal structure is *not* worth its indirection.

A6. (Lessons 14, 13) Why is the network boundary a place where TypeScript's static guarantees stop? Explain what a mapper/anti-corruption layer and runtime validation at the boundary buy you, and what goes wrong when DTOs leak through components.

A7. (Lessons 15, 16) Define state *ownership* (as distinct from location and access). Why is copying server data into local component state a design smell, and what is a *draft*? Name the typical owners of: form input, selected row, filter, fetched entity, authentication session.

A8. (Lessons 17, 18) Explain unidirectional data flow using the command / event / query vocabulary, and why traceability is the main architectural benefit. Why is the URL called the "spine" of an application, and which kinds of state belong in it?

A9. (Lesson 19) What does an error boundary catch and *not* catch? Explain boundary granularity as a blast-radius decision and how recovery (reset) works. Name the React 19 root-level callbacks for reporting and when each fires.

A10. (Lessons 20, 21) Describe the three token tiers and why theming remaps one of them. List the packaging facts that make a React component library safe to consume (`exports`, `sideEffects`, peers). When is "adopt a third-party library behind an adapter" better than building one?

A11. (Lessons 22, 23) What is a monorepo (and not)? Explain just-in-time vs compiled internal packages, the "duplicate React" failure, and how Conway's law and CODEOWNERS relate to module boundaries. What makes a `shared` package a dumping ground?

A12. (Lessons 24, 25, 27) Describe a safe approach to changing legacy code (characterization test, seam, strangler/parallel change) and how a ratchet keeps the migration from regressing. Then, for styling: state the three axes along which approaches differ and why "continuous values go in CSS variables".

Part B — Code and structure prediction (24 points; 3 each)

For each item state the result and justify it in 1–3 sentences. Assume React 19, StrictMode **off**, and a fresh mount unless stated. Scoring: 1 point for the correct result, up to 2 for the causal justification.

B1. (Lesson 19)

```
class Boundary extends React.Component<{ children: React.ReactNode }, { failed: boolean }> {
  state = { failed: false };
  static getDerivedStateFromError() { return { failed: true }; }
  render() { return this.state.failed ? <p>Oops</p> : this.props.children; }
}

function Boom(): React.ReactNode { throw new Error("x"); }

function App() {
  return (
    <>
      <Boundary>
        <header>H</header>
        <Boom />
      </Boundary>
      <footer>F</footer>
    </>
  );
}
```

(a) What text content is visible after the first render? (b) If `Boom` were instead `function Boom() { return <button onClick={() => { throw new Error("x"); }}>go</button>; }` and the user clicks, does `Oops` appear?

B2. (Lesson 09) Using Feature-Sliced Design's layer order `app → pages → widgets → features → entities → shared` (higher layers may import from lower ones), which of the following imports are **allowed**?

1. `features/cart/ui/CartButton.tsx` → `entities/product` (its public API)
2. `entities/product` → `features/cart`
3. `features/cart` → `features/checkout` (public API)
4. `pages/home` → `widgets/header`
5. `shared/ui` → `entities/user`
6. `features/cart/ui/CartButton.tsx` → `features/cart/model/cartStore.ts`

B3. (Lessons 19, 16)

```
const profilePromise = fetch("/api/me").then(r => { if (!r.ok) throw new Error("HTTP " +
r.status); return r.json() as Promise<{ name: string }>; });

function Profile() {
  const me = use(profilePromise);
  return <p>{me.name}</p>;
}

<ErrorBoundary fallback={<p>E</p>}>
  <Suspense fallback={<p>L</p>}>
    <Profile />
  </Suspense>
</ErrorBoundary>
```

`ErrorBoundary` is a correct error boundary. The request takes 1 s and the server responds **500**. What does the user see at t = 0 and at t = 1 s?

B4. (Lessons 15, 03)

```
function useTheme() {
  const [theme, setTheme] = useState<"light" | "dark">("light");
  return { theme, toggle: () => setTheme(t => (t === "light" ? "dark" : "light")) };
}
function Header()  { const { theme, toggle } = useTheme(); return <button onClick={toggle}>
{theme}</button>; }
function Sidebar() { const { theme } = useTheme(); return <aside>{theme}</aside>; }
```

Both are rendered under one parent. The user clicks the Header button once. What do `Header` and `Sidebar` display, and what ownership defect does this reveal?

B5. (Lesson 16)

```
function EditUser({ id }: { id: string }) {
  const user = useUserQuery(id);          // server-state hook; returns { name: string }
(already loaded)
  const [name, setName] = useState(user.name);
  return <input value={name} onChange={e => setName(e.target.value)} />;
}
```

`user.name` is `"Ann"` on mount. Without the user typing, a background refetch updates the cache so `user.name` becomes `"Anna"`. What does the input show? What if the parent changes `id` to another user without remounting `EditUser`?

B6. (Lesson 27)

```
:root          { --c: red; }
.theme-blue    { --c: blue; }
.badge         { color: var(--c, green); }
.other         { color: var(--missing, green); }
```

```
<p class="badge">1</p>
<div class="theme-blue"><p class="badge">2</p><p class="other">3</p></div>
```

What colour is each of the three paragraphs and why?

B7. (Lessons 05, 25, Phase 0 / 10) Native ES modules, entry point `main.ts`:

```
// a.ts
import { b } from "./b";
export const a = "A";
console.log(b);
// b.ts
import { a } from "./a";
export const b = "B" + a;
// main.ts
import "./a";
```

What happens when `main.ts` runs in a native-ESM environment? State which module's evaluation order matters and which architectural rule would have prevented it.

B8. (Lesson 21) `@acme/ui`'s `package.json` has `"exports": { ".": { "types": "./dist/index.d.ts", "import": "./dist/index.js" } }`. A consumer writes `import { secret } from "@acme/ui/dist/internal/secret.js"`. What happens at resolution time, and what architectural property does this demonstrate?

Part C — Debugging challenges (30 points; 6 each)

For each: (1) identify the symptom's root cause, (2) explain the mechanism, (3) give a correct fix, (4) say how you would verify it. Scoring: 1.5 points per item.

C1. The boundary that never shows (Lesson 19)

```
function Orders() {
  const [orders, setOrders] = useState<Order[]>([]);
  useEffect(() => {
    fetch("/api/orders")
      .then(r => r.json())
      .then(setOrders);
  }, []);
  return <ul>{orders.map(o => <li key={o.id}>{o.title}</li>)}</ul>;
}

<ErrorBoundary fallback={<p>Something went wrong</p>}>
  <Orders />
</ErrorBoundary>
```

When the API returns HTTP 500 with a JSON body `{"error":"db down"}`, users see... what? In another incident the API returned HTTP 502 with an HTML error page. What happened this time? Why does neither case show "Something went wrong" (or does one?), and how do you fix it properly?

C2. The cycle (Lessons 06, 09, 25)

```
// features/cart/index.ts
export * from "./model/cart";
export * from "./ui/CartButton";

// features/cart/ui/CartButton.tsx
import { useUser } from "@/features/user";

// features/user/index.ts
export * from "./model/user";

// features/user/model/user.ts
import { cartCount } from "@/features/cart";
export const useUser = () => ({ badge: cartCount() });
```

Symptom: unit tests that import `@/features/user` first pass; tests that import `@/features/cart` first fail with `Cannot access 'cartCount' before initialization` (or `useUser is not a function` depending on the toolchain). Production works "most of the time". Diagnose, fix, and name the rule that should have blocked it.

C3. The hook that only breaks in the app (Lessons 21, 22) A workspace has `apps/shop` (React 19.0) and `packages/ui` (compiled to `dist`, consumed by the shop). After the first release of the build pipeline, the shop crashes at the first `useState` call inside `Button` with `Cannot read properties of null (reading 'useState')` and a console message about "more than one copy of React" or "Invalid hook call". `packages/ui/package.json` lists `"dependencies": { "react": "^19.0.0" }`, and the lockfile resolves a different patch version under `packages/ui/node_modules`. Diagnose, fix, and describe two ways to detect this class of defect in CI.

C4. Two owners (Lessons 15, 16)

```
function ProfileForm({ id }: { id: string }) {
  const { data: user } = useUserQuery(id);          // cached server state
  const [form, setForm] = useState({ name: user.name, email: user.email });
  const save = useSaveUser();
  return (
    <form onSubmit={e => { e.preventDefault(); save.mutate({ id, ...form }); }}>
      <input value={form.name} onChange={e => setForm(f => ({ ...f, name: e.target.value }))}
/>
      <input value={form.email} onChange={e => setForm(f => ({ ...f, email: e.target.value
}))} />
    </form>
  );
}
```

Support reports: (1) after saving on another screen, this form shows stale values; (2) navigating from user 1 to user 2 through a link shows user 1's data in the form but saves under user 2's id. Diagnose both, name the ownership defect, and give a fix that preserves in-progress edits sensibly.

C5. The lint rule that protects only half the code (Lesson 25)

```
// eslint.config.js (flat config)
export default [
  {
    files: ["src/features/**/*.{ts,tsx}"],
    rules: {
      "no-restricted-imports": ["error", {
        patterns: [{ group: ["@/features/*/*"], message: "Import from the feature's public
API." }],
      }],
    },
  },
  {
    files: ["src/features/**/*.ts"],
    rules: {
      "no-restricted-imports": ["error", { paths: [{ name: "lodash", message: "Use native
methods." }] }],
    },
  },
];
```

Team observation: deep imports like `import { x } from "@/features/cart/model/internal"` are flagged in `.tsx` files but **not** in `.ts` files, and inside `features/cart` itself, files that import their own `@/features/cart/model/x` through the alias are flagged. List all defects and give a corrected configuration plus a verification approach.

Part D — Implementation (30 points; 10 each)

Write real, type-correct TypeScript/React. Rubrics are in the answer key; read them only after attempting.

D1. An error boundary with recovery and async propagation (Lesson 19) Implement:

1. `ErrorBoundary` (class component) with props `fallbackRender({ error, reset })`, optional `onError(error, info)` for reporting, and optional `resetKeys` (array; when any key changes while an error is displayed, the boundary resets automatically).
2. A hook `useErrorHandler()` returning a function that, when called from an event handler, effect or promise callback, causes the *nearest boundary* to render its fallback.
3. A short usage example fixing the `Orders` component of C1.

D2. A use case behind ports (Lessons 10, 11, 12, 13) Implement, for an e-commerce checkout:

1. A pure domain function `validateCart(cart): Result` (non-empty; every line has an integer quantity ≥ 1; money in integer minor units).
2. An application service `placeOrder` that depends on a `OrderGateway` port (owned by the application layer) and an idempotency-key generator, returning a discriminated-union result (`ok` or reasons: `empty`, `invalid`, `rejected`, `network`) and never throwing for expected failures.
3. A React hook `usePlaceOrder()` that gets the service from a context set at the composition root, exposing `{ run, status }`.
4. A unit test of the service using a hand-written fake gateway (no module mocking).

D3. Tokens, theming and layers (Lessons 20, 21, 27) Implement:

1. A CSS entry file that declares cascade layers `reset, tokens, base, components, utilities`, with tokens imported into the `tokens` layer.
2. A token file with primitive, semantic tokens, and a `[data-theme="dark"]` block that remaps **only semantic** tokens.
3. A `Button` component (TypeScript, React 19 `ref` as a prop) using CSS Modules with a closed `variant` union driven by `data-variant`, referencing only semantic tokens.
4. A `useTheme` hook (no flash on reload is *not* required) that sets `data-theme` on `<html>` without making any component re-render for colour changes, and a one-paragraph note on how you would prevent a flash of the wrong theme in an SSR setting.
5. Three lint rules (described, not configured in full) that keep the system healthy.

Part E — Architecture and reasoning (16 points; 4 each)

Answer in a structured way: **criteria → decision → accepted trade-offs → condition that would change the decision**. About 200–350 words each.

E1. (Lessons 01, 06, 07, 09, 23, 25) A product has 40 engineers in 5 teams working in one React codebase organised as `components/`, `hooks/`, `services/`, `utils/`. Cross-imports are everywhere, a "shared" folder holds 600 files, and every release has regressions caused by unrelated changes. Propose a target structure, the first three enforceable rules, a rollout plan that does not stop feature work, and one situation in which you would *not* restructure.

E2. (Lessons 15, 16, 17, 18) A back-office dashboard has: a filterable, sortable, paginated orders table; a detail drawer for the selected order; an edit form with unsaved changes; a user preference for column visibility; and a requirement that a colleague can open a link and see the same filtered view. Produce an **ownership map** (each datum: owner, lifetime, location, who may write) and justify at least two decisions with trade-offs.

E3. (Lessons 20, 21, 22, 27) Three product apps (two use runtime CSS-in-JS, one uses Sass) with 25 engineers share nothing but copy-pasted components; marketing wants one visual language in 9 months, and one app is moving to a Server Components framework. Propose how to package the shared UI (internal folder / workspace package / published), the styling strategy, token approach, governance, and the migration order. State what you would measure to decide whether the effort worked.

E4. (Lessons 24, 25) A 300,000-line app has class components, Redux, an in-house CSS-in-JS wrapper and almost no tests. The business will not allow a rewrite. Design a 12-month modernisation plan that prioritises risk reduction: how you create safety nets, which seams you cut first, how you sequence migrations, how fitness functions and ratchets are used, and what you explicitly **will not** do.

Answer Key

> Scoring convention: 1 point per bold key idea in Part A. A student's own wording is fine; points require the causal link where indicated.

Part A — Key ideas (3 points each: 1 per bold idea)

A1. (1) An architectural decision is one that **shapes the cost and shape of future change and is expensive to reverse** (module boundaries, state ownership, data flow, dependency direction), unlike a local design choice (a component's prop names) or implementation detail (a loop). (2) **Coupling** is how much a change in one unit forces changes in others; **cohesion** is how strongly the contents of a unit belong together; **high coupling and low cohesion produce change amplification** (one requirement edits many files). (3) **Over-separation adds indirection** (more files, layers, abstractions to navigate, "ravioli code"), raising cognitive load and hiding behaviour without reducing the reasons to change; separation should follow *real axes of change* and be introduced at the last responsible moment (01, 02). Full credit also accepts Conway's law/drift/fitness functions mentioned as supporting ideas.

A2. (1) A component's responsibility is a **contract: what it owns (state, effects, data, layout) and what it demands/offers (props, children, events)**; a "capability budget" and the prop surface expose over-responsibility (boolean-prop explosion, god components). (2) The presentational/container split separated

data-aware from view-only components, for reuse, testability, and design tooling (Storybook-style work); "smart/dumb" is discouraged because it **moralises a spectrum and is often applied mechanically**, creating pass-through wrapper layers; the useful distinction is *data awareness*, not intelligence. (3) **Custom hooks (and headless components, plus Server/Client Component boundaries) replaced wrapper containers**: logic extraction without an extra component layer; over-separation still costs (03, 04).

A3. (1) DIP: **high-level policy should not depend on low-level detail; both depend on abstractions, and the abstraction is owned by the high-level (consuming) side** (the application defines the `OrderGateway` interface it needs; the HTTP adapter implements it). (2) React mechanisms (any three): **props/arguments, children/slots/render props, context injection, hook injection (via a provider), factory/closure injection, module substitution at build time**. (3) A hidden service locator **makes dependencies implicit, hard to see in types and tests, and failures appear at runtime**; explicit injection keeps seams visible and substitutable. The composition root is **the single place near the application entry where concrete implementations are chosen and wired**; a correct statement of it counts toward idea 1 or 3.

Scoring note for A3: award one point each for ideas 1, 2 (at least three mechanisms) and 3.

A4. (1) Layer-organisation groups by *technical role* (components, hooks, services), so **one feature change touches many folders and there is no ownership boundary**; feature organisation groups by *axis of change*, giving **locality of change and a natural ownership/test/delete unit**; both can fail (layer: scattering; feature: duplication, cross-feature tangles), so the promotion rule and a shared kernel matter. (2) FSD import rule: **a module may import only from layers strictly below it; slices on the same layer may not import each other** (shared cross-slice needs go down a layer or use the documented cross-import mechanism ⚠ verify the current spec); within a slice, any segment may import its own siblings. (3) A public API (`index.ts`) **is the slice's contract: other code imports only through it, enabling internal refactors and lintable boundaries**; its **cost** is barrel-file effects on dev-server speed/tree shaking and the risk of exporting too much or creating cycles (06, 07, 09).

A5. (1) Domain logic belongs in **plain TypeScript modules (pure functions, reducers, value objects) that import neither React nor transport libraries**, so it is testable without rendering and reusable across UI technologies/server. (2) Distinction: **domain** = business rules independent of any screen; **application** = use-case orchestration (steps, side effects, ports); **presentation** = view models, interaction state, formatting — and the **line is drawn by "would this rule exist if the UI were replaced?"** (3) A **port is an interface owned by the application describing a capability it needs or offers; an adapter implements it for a concrete technology** (HTTP, storage, router, notifications). (4) Hexagonal/clean structure is not worth it for **thin CRUD screens with no meaningful rules, short-lived projects, or tiny teams**, where indirection costs more than the substitutability it buys; DDD tactics only where the domain is complex (08, 10, 12, 13). Credit three of four.

A6. (1) **TypeScript types are erased at runtime; data arriving over the network is untrusted and may differ from its declared type** (API version drift, partial failures, null fields), so `as T` on a `fetch` result is a claim, not a check. (2) A **mapper/anti-corruption layer translates DTOs into domain/view models at a single place** (renaming, defaulting, parsing dates and money, normalising errors), and **runtime validation at the boundary** (schema parsing) turns silent data corruption into an early, located error. (3) **When DTOs leak into components, API shape changes ripple through the UI, null/format handling is duplicated**

in many components, and tests must construct transport-shaped data; centralising access in gateways/data-access modules restores one place to change (13, 14).

A7. (1) Ownership is **who has write authority and lifecycle responsibility for a datum** (single source of truth); *location* is where it is stored, *access* is who may read it; confusing these produces prop-drilling vs global-store debates that miss the real question. (2) Copying server state into local state creates **a second owner that diverges (staleness, no invalidation, stale initial values)**; server state is a **cached replica** with keys, staleness and invalidation. A **draft** is **local, user-owned, uncommitted edits** layered over (not copied from) the entity, discarded or committed on save. (3) Typical owners: form input → the form component (draft); selected row → URL or the owning list/screen depending on shareability; filter → URL search params when shareable/bookmarkable; fetched entity → the server-state cache; auth session → server/session boundary exposed through a narrow provider (15, 16).

A8. (1) Unidirectional flow: **commands/intent flow up or out (events from the UI become commands), state changes through one owner, and derived/query data flows down**; events are facts that happened; queries read. (2) The benefit is **traceability: for any rendered value you can find the single write path, making bugs, replay, and review tractable**; cycles, bidirectional sync, and ad-hoc event buses break it. (3) The URL is the spine because **it is the only state a user can share, bookmark, refresh and navigate with (back/forward)**; it should hold **location and shareable view state (route params, filters, sort, pagination, tab, selected entity id)**, not ephemeral UI state or secrets (17, 18).

A9. (1) Boundaries catch **errors thrown while rendering, in lifecycle methods, and in constructors of their descendants**; they do **not catch event-handler errors, asynchronous code (promises, timers), server-side rendering errors, or errors thrown in the boundary itself** (these need other propagation, e.g. storing error in state and rethrowing during render, or root callbacks). (2) Granularity trades **blast radius vs fallback consistency**: a root boundary protects the app but loses the screen; per-route/feature/widget boundaries keep the rest alive; recovery uses **reset (clearing the error state, often keyed to route/props/retry) so descendants remount**. (3) React 19 root options: `onCaughtError` **(errors caught by a boundary),** `onUncaughtError` **(errors not caught by any boundary),** `onRecoverableError` **(errors React recovers from automatically, e.g. hydration mismatches)**; used for reporting with component stacks (19). Boundaries themselves still require a class component (verify against current docs for any change).

A10. (1) Token tiers: **primitive (raw values), semantic (roles), component (optional local aliases)**; **theming remaps the semantic tier**, so components referencing only semantic tokens need no change and, with CSS custom properties, React does no work on theme change. (2) Packaging facts: `exports` **map as the only door (encapsulation,** `types` **condition), ESM output,** `sideEffects` **listing CSS so imports aren't dropped,** `react` **/** `react-dom` **as peerDependencies (and external in the build) to avoid duplicate React and context breaks**; also types emitted, no source shipped. (3) Adopt-behind-adapter is better **for small/medium teams, internal tools, or short-lived products where time-to-value and vendor-tested a11y outweigh brand specificity; the adapter keeps replacement possible** (20, 21).

A11. (1) A monorepo is **one repository containing multiple independently described packages/apps with explicit dependency edges, not "one big app"**. (2) *Just-in-time* packages export source consumed and compiled by the app's toolchain (simple, fast feedback, requires consumers to transpile); *compiled* packages ship built output with types (decoupled, a build step). **Duplicate React** occurs when a package bundles/declares React as a dependency so two instances exist, breaking hooks/context; fix with peer

dependencies, dedupe/externals. (3) **Conway's law: system structure mirrors communication structure; module/package boundaries should align with team ownership, expressed through CODEOWNERS and contracts**; `shared` becomes a dumping ground when **it accepts anything without an owner, promotion criteria or domain knowledge, so everything depends on it and changes to it ripple everywhere** (22, 23).

A12. (1) Legacy change: **characterization tests (pin current behaviour), seams (places to change behaviour without editing in place), strangler/parallel-change (introduce new path, migrate callers, remove old), anti-corruption layers at the boundary of old/new**; avoid big-bang rewrites. (2) A **ratchet** records current violation counts/baselines and fails if they *increase*, so **new code obeys the rule while old violations shrink opportunistically**; fitness functions make architecture rules executable. (3) Styling axes: **authoring syntax, scoping mechanism, and when CSS is produced (author/build/run time)**; continuous values (positions, widths from data) have **unbounded cardinality**, so generating a class per value explodes rules/invalidations; **a CSS variable lets one rule read a per-element value without injection** (24, 25, 27).

Part B — Answers (3 points each: result 1, reasoning 1–2)

B1. (a) Visible text: `Oops` **and** `F`. `Boom` throws during render; the nearest boundary's `getDerivedStateFromError` sets `failed`, and the boundary renders its fallback *instead of its whole child subtree* (so `H` disappears too); the sibling `footer` outside the boundary is unaffected. (b) **No,** `Oops` **does not appear.** Error boundaries do not catch errors thrown in event handlers; the handler runs outside render, the UI remains `go`, and the error is reported as an uncaught error (browser console / `onUncaughtError` in React 19). Fix paths: try/catch + state, or propagate via the pattern in D1.

B2. Allowed: **1, 4, 6.** (1) features → entities: downward, through public API: allowed. (2) entities → features: upward: forbidden. (3) features → features: same layer, different slice: forbidden (extract downward, compose in a higher layer, or use the documented cross-import exception ⚠ verify spec). (4) pages → widgets: downward: allowed. (5) shared → entities: upward: forbidden (shared is the lowest layer). (6) Within one slice, segments may import siblings: allowed (the "public API only" rule applies to *outside* consumers).

B3. t = 0: `L` (`use` suspends on the pending promise; the nearest Suspense shows its fallback). t = 1 s: `E`. The `fetch` resolves with a 500, the `.then` throws `Error("HTTP 500")`, the promise rejects, `use` throws the rejection reason during render of `Profile`, and the nearest error boundary *above the throwing component* is the outer `ErrorBoundary`. Note the boundary is outside Suspense, so the Suspense fallback is replaced by the error fallback. (A caveat for completeness: creating the promise at module scope as shown is for demonstration; in applications the promise must be cached/stable across renders; see Phase 9 / 05.)

B4. After one click: **Header shows** `dark`**; Sidebar still shows** `light`**.** Each `useTheme()` call creates its own `useState` slot in its own component instance; hooks share *logic*, not *state*. The defect: **the datum has two owners (or none)**; the theme should be owned once (a provider/context or the DOM attribute/external store) and read by both components (15; Phase 3 / 15).

B5. The input still shows `Ann`. `useState(user.name)` uses its argument only on the *first* render; later renders ignore it, so the refetched `"Anna"` never reaches the input (a second owner that was seeded from

the first). If the parent changes `id` without remounting, the input still shows the **previous user's** name for the same reason. Fixes: derive from the server value unless the user has edited (`draft ?? user.name`), or remount with `key={id}` and treat the form's local state as a draft (16, Phase 1 / 16 state preservation and reset).

B6. (1) **red**: no `.theme-blue` ancestor; `--c` inherits `red` from `:root`. (2) **blue**: custom properties inherit; the `.theme-blue` ancestor redefines `--c` for its subtree. (3) **green**: `--missing` is not defined anywhere, so the fallback value `green` is used. (The fallback in `.badge` is irrelevant for (1) and (2) because `--c` is defined.) Theming via custom properties works by redefining them on an ancestor (27, 20).

B7. A `ReferenceError` (`Cannot access 'a' before initialization`) is thrown while evaluating `b.ts`. Evaluation order: `main` imports `a`; `a`'s dependency `b` is evaluated first (imports are hoisted and dependencies run before the importer's body); `b`'s body reads the `a` binding, which exists but is in the temporal dead zone because `a.ts`'s body has not run. Only a *cycle* makes this possible; rule: **acyclic dependencies / dependency direction (no module may be reached from its own dependencies)**, enforced with a cycle check (05, 25). Bundlers may transform ESM differently, so symptoms can be `undefined` instead of a throw; the cycle is the defect either way.

B8. Resolution **fails**: the bundler/Node reports that the subpath `./dist/internal/secret.js` is **not defined by the package's** `exports` (Node: `ERR_PACKAGE_PATH_NOT_EXPORTED`). This demonstrates **hard encapsulation**: `exports` makes the package's public API the only importable surface, unlike `main`, which exposes the whole file tree. TypeScript with modern resolution also rejects the import (21, 22).

Part C — Debugging (6 points: 1.5 per item)

C1.

- **Cause (1.5):** Errors in `useEffect`'s promise chain are asynchronous and happen **outside React's render**, so **error boundaries do not see them**. In the HTTP 500 + JSON `{"error":"db down"}` case, `r.json()` succeeds, `setOrders({error:"db down"})` stores an object, and the **next render** calls `orders.map` on an object → `TypeError: orders.map is not a function` thrown **during render**, which **is** caught: users see "Something went wrong" (by accident, with a misleading cause). In the 502 HTML case, `r.json()` rejects (`SyntaxError`) in the promise chain → **unhandled promise rejection**: no re-render, no boundary; users see an **empty list forever** and the error appears only in the console/global handlers.
- **Mechanism (1.5):** Boundaries catch errors thrown while React renders/updates the tree; promise callbacks run later from the microtask queue; React cannot associate them with a component render. Also, nothing checks `r.ok` or the response shape.
- **Fix (1.5):** Check `r.ok`; validate/map the payload at the boundary; keep `status`/`error` state; either render an error UI locally for *expected* failures, or **rethrow into render** for unexpected ones (`if (error) throw error;` or the `useErrorHandler` of D1); handle abort/cleanup of the effect; better, use a server-state hook/loader that integrates with Suspense and error boundaries (19, 16).
- **Verification (1.5):** Tests with a mocked 500/JSON, 502/HTML, network failure: assert the fallback or local error UI shows; assert `onError`/reporting is called; check console for no unhandled rejection.

C2.

- **Cause (1.5):** A **dependency cycle between features** through their barrels: `cart → user → cart`. Module evaluation order depends on which module is imported first, so the binding is uninitialised (TDZ) or `undefined` when read at module-evaluation time (which binding is uninitialised depends on the entry module and the toolchain).
- **Mechanism (1.5):** ESM evaluates dependencies before dependents; in a cycle one module must run before one of its dependencies has finished. `export *` through barrels widens the cycle (every export of each barrel becomes an edge), making it fragile and entry-order dependent (works in production if the bundle happens to order modules favourably).
- **Fix (1.5):** Break the cycle at the weakest edge, structurally: features must not import each other. Options: (a) move the *badge composition* to a higher layer (page/widget) that imports both public APIs and passes `cartCount` to the user UI via props/slot; (b) extract the shared concept (e.g., cart summary) down into an `entities`/shared kernel module both may import; (c) invert: `user` exposes an extension point (slot/provider) that `cart` fills at the composition root. Replace `export *` with explicit exports.
- **Rule (stated in task): Same-layer slices/features may not import each other; plus "no cycles"** enforced by a graph/cycle tool and boundary lint in CI (with a ratchet if existing cycles exist) (09, 25).
- **Verification (1.5):** Run the cycle detector (no cycle through the two features); run the tests with each module as the first import; add a boundary-lint canary.

C3.

- **Cause (1.5): Two copies of React**: `packages/ui` has its own React (declared as a `dependency`, resolving differently or bundled into `dist`) and the app has another. The library's `useState` comes from copy A of `react`, but the renderer rendering the component is copy B's `react-dom`, so copy A has no active hook dispatcher (conceptually its internal "current dispatcher" is unset; internal names are implementation details) → `Cannot read properties of null` / "Invalid hook call".
- **Mechanism (1.5):** Hooks rely on a module-level "current dispatcher" set by the renderer instance; a library using a different `react` module instance sees none. (Context has the same issue with distinct `createContext` objects.)
- **Fix (1.5):** Move `react` / `react-dom` to `peerDependencies` (plus `devDependencies` for local work); mark them `external` in the build; ensure the lockfile resolves a single instance (`pnpm why react` / `npm ls react`; `dedupe`); for linked dev, alias `react` to the app's copy.
- **Detection/verification (1.5):** (1) A CI pack-and-install test: install the tarball in a fixture app, build it, and assert the bundle contains exactly one React (e.g. search for a unique React string) or render a component that uses a hook; (2) a lockfile/workspace check that fails when more than one version of `react` resolves; also package lint tooling (⚠ verify tool names) and `exports`/peer review (21, 22).

C4.

- **Cause (1.5):** `useState({ name: user.name, … })` **copies the server replica into a second owner at mount time**, never updating from later server changes, and **never resetting when** `id` **changes** because `ProfileForm` is not remounted (same element type and position; state preserved).
- **Mechanism (1.5):** (1) Initial state is read once; the cache can update (after saves elsewhere) but the form's copy remains. (2) With a new `id`, React preserves the component's state slot, so `form` still

holds user 1's values while `save.mutate({ id, ...form })` uses the new `id` → **data corruption** (user 1's data written to user 2).

- **Fix (1.5):** Treat local state as a **draft** over the entity: store only the *changed* fields (`const [draft, setDraft] = useState<Partial<Form>>({})`), render `draft.name ?? user.name`, and **reset the draft on `id` change** by giving the form `key={id}` at the call site (or inside via key on an inner component). Decide what happens when the server entity changes under an *edited* field (keep draft, show a "changed remotely" indicator) and clear the draft after successful save. Optionally use the form library's `values`/reset mechanism if one is used.
- **Verification (1.5):** Tests: render with user 1, edit, rerender with user 2: form shows user 2 and no draft; update cache while untouched: form shows the new value; update cache while edited field: draft preserved; save payload uses the right id.

C5.

- **Defects (1.5 for identifying 2+; 1.5 mechanism):**
  1. **Rule replacement, not merge.** In flat config, when several matching config objects set the same rule, **later objects replace the earlier rule's options** (they do not merge `patterns` and `paths`). For `.ts` files under `src/features/**` the second block replaces the first, so the deep-import pattern is **not applied to `.ts` files**; `.tsx` files only match the first block, so they are protected: inconsistent enforcement.
  2. **False positives:** the pattern `@/features/*/*` forbids *any* two-segment path, including a feature importing **its own** internals via the alias; it should only forbid *other* features' internals (requires per-feature config or a boundary plugin/graph tool with "same slice" semantics).
  3. **Relative path imports** (`../../cart/model/internal`) are not matched by an alias-only pattern.
  4. No **canary tests**, so the half-enforcement went unnoticed; severity/ `--max-warnings` irrelevant here since the rule is `error`, but note unlinted file types (`.mts`, `.js`).
- **Fix (1.5):** Merge into one block per file set with both `patterns` and `paths`; or generate per-feature blocks that forbid other features' internals (`group: ["@/features/*/*", "!@/features/cart/*"]` is *not* a per-feature exception: executed on ESLint 9.39.5, the `!` negation does re-allow `@/features/cart/*` (and anything beneath it) and still flags `@/features/fleet/internal`, but it is one exception for every file in the block, so cart's rule would also let *fleet* import cart internals; prefer per-feature blocks or a boundary/graph tool); include relative-path patterns; cover all extensions in `files`.

```
export default [
  {
    files: ["src/features/**/*.{ts,tsx}"],
    rules: {
      "no-restricted-imports": ["error", {
        paths: [{ name: "lodash", message: "Use native methods." }],
        patterns: [
          { group: ["@/features/*/*", "**/features/*/*"], message: "Import from the feature's
public API." },
        ],
      }],
    },
  },
  // per-feature allowance: features may import their own internals via relative paths only
];
```

(Evaluation: any config that merges options and addresses own-feature false positives, with a note to verify against current ESLint docs, earns full credit.)

- **Verification (1.5):** Canary tests using ESLint's Node API on virtual files for both `.ts` and `.tsx`, aliased and relative, own-feature and other-feature; CI runs them; also run the linter against a deliberately violating fixture and assert failure (25).

Part D — Implementation key (10 points each)

D1. Error boundary — rubric

Points	Criterion
3	Class component with `static getDerivedStateFromError` storing the error and rendering `fallbackRender({ error, reset })`; correct typing (`unknown` normalised to `Error`)
2	`componentDidCatch(error, info)` calls `onError` (reporting side effect belongs here, not in `getDerivedStateFromError`)
2	`resetKeys` comparison in `componentDidUpdate` resets only when an error is currently displayed and keys changed (shallow compare, length-aware)
2	`useErrorHandler` rethrows during render (state updater throwing, or storing error in state then throwing in render)
1	Usage example fixing `Orders` (check `r.ok`, catch, call handler)

```tsx
import { Component, useCallback, useState, type ErrorInfo, type ReactNode } from "react";

type FallbackArgs = { error: Error; reset: () => void };
type Props = {
  children: ReactNode;
  fallbackRender: (args: FallbackArgs) => ReactNode;
  onError?: (error: Error, info: ErrorInfo) => void;
  resetKeys?: readonly unknown[];
};
type State = { error: Error | null };

const toError = (e: unknown): Error => (e instanceof Error ? e : new Error(String(e)));
const keysChanged = (a: readonly unknown[] = [], b: readonly unknown[] = []) =>
  a.length !== b.length || a.some((v, i) => !Object.is(v, b[i]));

export class ErrorBoundary extends Component<Props, State> {
  state: State = { error: null };

  static getDerivedStateFromError(error: unknown): State {
    return { error: toError(error) };            // pure: just record the error
  }

  componentDidCatch(error: unknown, info: ErrorInfo) {
    this.props.onError?.(toError(error), info);  // side effect: reporting
  }

  componentDidUpdate(prev: Props) {
    if (this.state.error && keysChanged(prev.resetKeys, this.props.resetKeys)) this.reset();
  }

  reset = () => this.setState({ error: null });

  render() {
    return this.state.error
      ? this.props.fallbackRender({ error: this.state.error, reset: this.reset })
      : this.props.children;
  }
}

export function useErrorHandler() {
  const [, setState] = useState<void>();
  return useCallback((error: unknown) => {
    setState(() => { throw toError(error); });   // the updater runs during render → the boundary catches it
  }, []);
}
```

Usage:

```
function Orders() {
  const [orders, setOrders] = useState<Order[]>([]);
  const handleError = useErrorHandler();
  useEffect(() => {
    const ac = new AbortController();
    fetch("/api/orders", { signal: ac.signal })
      .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json() as
Promise<Order[]>; })
      .then(setOrders)
      .catch(e => { if (!ac.signal.aborted) handleError(e); });
    return () => ac.abort();
  }, [handleError]);
  return <ul>{orders.map(o => <li key={o.id}>{o.title}</li>)}</ul>;
}
```

Edge cases for full marks: thrown non-`Error` values; reset while the same error repeats (infinite loop risk if `resetKeys` changes every render: keys must be stable); the boundary cannot catch its own `fallbackRender` errors (nest another boundary above); `reset` should remount children (it does: the subtree is recreated when `state.error` clears); StrictMode double-invokes the updater in dev (harmless). Production code often uses a library; this is for understanding.

D2. Use case behind ports — rubric

Points	Criterion
2	`validateCart` pure; no React/IO; integer minor-unit money; returns a discriminated union or list of issues
2	`OrderGateway` port defined **in the application layer** (consumer-owned); service takes dependencies as arguments (no imports of fetch/axios)
2	Result type models expected failures; unexpected errors map to `network` / `rejected` without throwing; idempotency key generated once per attempt (reused on retry)
2	`usePlaceOrder` obtains the service from context set at the composition root; hook holds status state; no domain logic in the hook
2	Test with a hand-written fake (success, rejection, network failure, empty cart); asserts the gateway call received the idempotency key

```ts
// domain/cart.ts
export type Line = { sku: string; quantity: number; unitPriceMinor: number };
export type Cart = { currency: string; lines: readonly Line[] };
export type CartIssue = "empty" | "invalid-quantity" | "invalid-price";

export function validateCart(cart: Cart): { ok: true } | { ok: false; issue: CartIssue } {
  if (cart.lines.length === 0) return { ok: false, issue: "empty" };
  for (const l of cart.lines) {
    if (!Number.isInteger(l.quantity) || l.quantity < 1) return { ok: false, issue: "invalid-quantity" };
    if (!Number.isInteger(l.unitPriceMinor) || l.unitPriceMinor < 0) return { ok: false, issue: "invalid-price" };
  }
  return { ok: true };
}

// application/placeOrder.ts
import { validateCart, type Cart } from "../domain/cart";

export type OrderGateway = {
  create(input: { cart: Cart; idempotencyKey: string }): Promise<{ orderId: string }>;
};
export class GatewayRejected extends Error {}   // adapter throws this for 4xx business rejections

export type PlaceOrderResult =
  | { ok: true; orderId: string }
  | { ok: false; reason: "empty" | "invalid" | "rejected" | "network" };

export type PlaceOrderDeps = { orders: OrderGateway; newKey: () => string };

export const makePlaceOrder = ({ orders, newKey }: PlaceOrderDeps) =>
  async (cart: Cart, key: string = newKey()): Promise<PlaceOrderResult> => {
    const v = validateCart(cart);
    if (!v.ok) return { ok: false, reason: v.issue === "empty" ? "empty" : "invalid" };
    try {
      const { orderId } = await orders.create({ cart, idempotencyKey: key });
      return { ok: true, orderId };
    } catch (e) {
      return { ok: false, reason: e instanceof GatewayRejected ? "rejected" : "network" };
    }
  };

// ui/PlaceOrderProvider.tsx (composition root supplies the real service)
import { createContext, use, useCallback, useRef, useState, type ReactNode } from "react";
type Service = ReturnType<typeof makePlaceOrder>;
```

```tsx
const Ctx = createContext<Service | null>(null);
export const PlaceOrderProvider = ({ service, children }: { service: Service; children:
ReactNode }) =>
  <Ctx value={service}>{children}</Ctx>;

export function usePlaceOrder() {
  const service = use(Ctx);
  if (!service) throw new Error("PlaceOrderProvider missing");
  const [status, setStatus] = useState<"idle" | "pending" | PlaceOrderResult>("idle");
  const keyRef = useRef<string | undefined>(undefined);   // reuse key across retries of the
same attempt
  const run = useCallback(async (cart: Cart) => {
    setStatus("pending");
    keyRef.current ??= crypto.randomUUID();               // created once per attempt, kept
across failed retries
    const result = await service(cart, keyRef.current);
    if (result.ok) keyRef.current = undefined;            // success consumes the key
    setStatus(result);
    return result;
  }, [service]);
  return { run, status };
}
```

Notes for graders: retaining the idempotency key across failed attempts (so a retry after a network error cannot create a duplicate order) is the sophisticated part; a simpler solution that generates a new key per call is acceptable only if the answer states the double-order risk when a response is lost. The `use(Ctx)` form is React 19 (`useContext` is equally correct). Domain logic must stay out of the hook. Test sketch:

```
import { describe, expect, it } from "vitest";
const cart = { currency: "EUR", lines: [{ sku: "a", quantity: 1, unitPriceMinor: 500 }] };
const fake = (impl: OrderGateway["create"]): OrderGateway => ({ create: impl });

it("passes the idempotency key and returns the order id", async () => {
  const seen: string[] = [];
  const place = makePlaceOrder({ orders: fake(async i => { seen.push(i.idempotencyKey); return
{ orderId: "o1" }; }), newKey: () => "k1" });
  expect(await place(cart)).toEqual({ ok: true, orderId: "o1" });
  expect(seen).toEqual(["k1"]);
});
it("maps network failure without throwing", async () => {
  const place = makePlaceOrder({ orders: fake(async () => { throw new TypeError("offline");
}), newKey: () => "k" });
  expect(await place(cart)).toEqual({ ok: false, reason: "network" });
});
it("rejects empty carts without calling the gateway", async () => {
  let called = false;
  const place = makePlaceOrder({ orders: fake(async () => { called = true; return { orderId:
"x" }; }), newKey: () => "k" });
  expect(await place({ ...cart, lines: [] })).toEqual({ ok: false, reason: "empty" });
  expect(called).toBe(false);
});
```

D3. Tokens, theming, layers — rubric

Points	Criterion
2	Layer order declared first; tokens imported/assigned to the `tokens` layer; component CSS in `components` layer
2	Tokens: primitive + semantic tiers; dark block remaps **semantic** tokens only; sensible naming by role
2	`Button`: closed `variant` union, `data-variant`, semantic tokens only, `type="button"` default, `ref` as a prop (no `forwardRef`), `:focus-visible` style
2	`useTheme` sets `document.documentElement.dataset.theme`; components do not subscribe for colour (no re-render for theme); persistence guarded with try/catch
1	Flash-prevention note: inline head script / cookie-based attribute on server render / `prefers-color-scheme` CSS before hydration; hydration mismatch caveat
1	Three lint rules (e.g. no raw colours or `--ref-*` in component CSS; no `!important` outside utilities layer; ban `className` / `style` passthrough or deep imports into the system)

```
/* app/styles/index.css */
@layer reset, tokens, base, components, utilities;
@import "./reset.css" layer(reset);
@import "@acme/ui/tokens.css" layer(tokens);
```

```
/* tokens.css */
:root {
  --ref-blue-600: #1f5fd1; --ref-blue-400: #5b8def; --ref-neutral-0: #fff; --ref-neutral-900:
#111827; --ref-neutral-950: #0b1220;
  --color-surface-default: var(--ref-neutral-0);
  --color-text-default: var(--ref-neutral-900);
  --color-action-primary: var(--ref-blue-600);
  --color-on-action: var(--ref-neutral-0);
}
[data-theme="dark"] {
  --color-surface-default: var(--ref-neutral-950);
  --color-text-default: #e5e7eb;
  --color-action-primary: var(--ref-blue-400);
  --color-on-action: var(--ref-neutral-950);
}
```

(If tokens are imported with `layer(tokens)` the entire file, including the `[data-theme]` block, lives in the `tokens` layer; consistent.)

```
// Button.tsx
import type { ComponentPropsWithRef } from "react";
import styles from "./Button.module.css";
type Variant = "primary" | "secondary";
export function Button({ variant = "primary", type = "button", ...rest }:
  Omit<ComponentPropsWithRef<"button">, "className" | "style"> & { variant?: Variant }) {
  return <button {...rest} type={type} data-variant={variant} className={styles.root} />;
}
```

```
/* Button.module.css */
@layer components {
  .root { padding: var(--space-2, 8px) var(--space-4, 16px); border: 0; border-radius: var(--
radius-md, 6px); cursor: pointer; }
  .root[data-variant="primary"]   { background: var(--color-action-primary); color: var(--
color-on-action); }
  .root[data-variant="secondary"] { background: var(--color-surface-default); color: var(--
color-text-default); outline: 1px solid currentColor; }
  .root:focus-visible { outline: 2px solid var(--color-action-primary); outline-offset: 2px; }
}
```

```tsx
// useTheme.ts
import { useCallback, useSyncExternalStore } from "react";
type Theme = "light" | "dark";
const listeners = new Set<() => void>();
const subscribe = (cb: () => void) => { listeners.add(cb); return () => listeners.delete(cb);
};
const read = (): Theme => (document.documentElement.dataset.theme === "dark" ? "dark" :
"light");
export function useTheme() {
  const theme = useSyncExternalStore(subscribe, read, () => "light" as Theme);
  const setTheme = useCallback((next: Theme) => {
    document.documentElement.dataset.theme = next;
    try { localStorage.setItem("theme", next); } catch { /* ignore */ }
    listeners.forEach(l => l());
  }, []);
  return [theme, setTheme] as const;
}
```

Flash note (model answer): a tiny synchronous script in `<head>` (or server-set `data-theme` from a cookie) sets the attribute before first paint; with SSR, rendering a theme-dependent *label* from client-only state causes hydration mismatch, so use the server snapshot (`"light"`) and update after hydration, or avoid rendering theme-dependent text. ⚠ verify framework-specific guidance. *Lint rules:* (1) stylelint: disallow hex/rgb/hsl literals and `--ref-*` usage in `**/*.module.css` outside the tokens package; (2) `declaration-no-important` except in the utilities layer files; (3) ESLint `no-restricted-imports` forbidding deep imports into `@acme/ui/*` internals and the headless library outside the system.

Part E — Reasoning key (4 points each: ≈1 for decision, 1.5 for criteria, 1.5 for trade-offs and counter-case)

E1. Evaluation criteria. A strong answer: (a) *Criteria:* team ownership alignment (Conway), axis of change, regression blast radius, migration cost, enforceability, team familiarity. (b) *Target:* feature/slice-based modular structure (e.g. FSD-like or "app / features / entities / shared" with public APIs), with `shared` restricted to domain-free utilities and UI primitives and an owner for each feature. (c) *First three rules:* (1) shared/lower layers never import features; (2) cross-feature imports only through public API (and same-layer features don't import each other); (3) no new cycles. (d) *Rollout:* baseline/ratchet the current violations; migrate along change hotspots (files that change often together); CODEOWNERS per feature; move code by feature when touched ("boy-scout"), characterization tests where behaviour is unclear; track cycle count and cross-feature deep imports on a dashboard; avoid a big-bang folder shuffle. (e) *Don't restructure when:* team ≤ 6, the product is near end-of-life, or regressions come mostly from missing tests/unclear requirements rather than structure (fix the real cause first). Example point allocation: decision 1; criteria 1.5; rollout/trade-offs (ossification, barrel costs, naming debates, temporary duplication) 1.5.

E2. Example ownership map (any equivalent well-reasoned map earns full marks):

Datum	Owner	Location	Lifetime	Writers
Filters, sort, page	URL (router)	search params	until navigation	filter UI via navigation/ `setSearchParams`
Orders list (rows)	Server-state cache	query keyed by filter params	staleness policy	cache layer (refetch/invalidate)
Selected order id	URL (path or `?order=`) so the link reproduces the view	route	navigation	row click / close action
Order entity (detail)	Server-state cache	query keyed by id	staleness policy	mutations → invalidate
Edit draft	Edit form component (draft only the changed fields)	local state keyed by order id	until save/cancel/navigate	form inputs
Unsaved-changes guard	Router blocker + form dirty flag	router + form	form lifetime	form
Column visibility	User preference store (local persistence, possibly synced to server)	external store/localStorage	long-lived	settings UI

Datum	Owner	Location	Lifetime	Writers
Decisions to justify: URL for shareable state (trade-off: URL noise, serialisation, history entries for every keystroke, so debounce text filters and `replace` instead of `push`); drafts separate from the entity (trade-off: merge conflicts when the server entity changes during editing); column visibility **not** in the URL (personal preference, not part of the shareable view) unless the product requires it. Counter-case: for a tiny internal tool, local component state for filters is acceptable if shareable links are not required. Deduct if the answer duplicates server data in a client store without reason, or places drafts in the global store without justification.				

E3. Evaluation criteria. Must contain: (a) *Packaging:* workspace package (JIT or compiled) in a monorepo if apps can live in one repo, published package if repos/teams must upgrade independently; start in-repo, publish later at trigger events; adapter strategy if adopting a third-party library. (b) *Styling:* tokens as CSS custom properties first (decouples approach), static CSS for the shared package (CSS Modules or zero-runtime) because RSC and strict CSP are in play, layers to let the old CSS-in-JS and Sass coexist during migration; do not demand all apps migrate at once. (c) *Migration order:* tokens and Button/Input/Typography first (highest reuse), the app moving to RSC converts its server-rendered shells first; freeze new runtime-styled components via lint; ratchet to zero. (d) *Governance:* a small core team plus hybrid contribution model, CODEOWNERS, a11y and visual tests as release gates, semver and deprecation policy. (e) *Metrics:* share of UI built from system components, count of raw colour literals, JS bundle share of styling runtime, hydration/INP/LCP for the RSC app, a11y violations, time-to-contribute,

version skew across apps. (f) *Counter-case:* if the three apps are scheduled for consolidation or retirement, or the brand unification is political rather than technical, a token-only first phase might suffice.

E4. Evaluation criteria. Expected elements: (1) *Safety nets first:* characterization/golden-master tests around critical flows (checkout, auth), end-to-end smoke tests, error boundaries and error reporting to discover real failures, feature flags for switching; (2) *Seams:* cut at I/O boundaries (API access into gateways), routing, and state stores before touching components; anti-corruption layers between old and new; (3) *Sequencing:* leaf components and new features on the new path (strangler), class → function components opportunistically, Redux → server-state + local/URL state by slice (starting with server data), styling via tokens + new components in CSS Modules (ratchet away the wrapper), codemods for mechanical changes; (4) *Fitness functions and ratchets:* cycle count, deep imports, count of class components, count of CSS-in-JS imports, bundle size budget, all non-increasing; exemptions with expiry; (5) *Governance:* ADR for each migration, ownership per area, weekly migration dashboard, explicit capacity (e.g. 15–20%) rather than heroic rewrites; (6) *Will not:* big-bang rewrite, simultaneous migration of state + routing + styling in the same module, restructuring folders purely for aesthetics, migrate code about to be deleted, adopt tools without measuring baselines. Counter-case: if the product is being sunset in 18 months, containment (tests around hotspots, no migration) is the right plan.

Scoring and Promotion

Rubric by category

Explanation (36 pts). A1–A12: 1 point per key idea stated in the student's own words **with the causal link**. Restating terms without why/how earns nothing (e.g. "ports and adapters decouple things" without saying who owns the port earns 0 for that idea). No deduction for omitted exact names of tools, plugin options or ESLint rule names.

Score	Interpretation
31–36	Can teach the model; ready to review others' architecture
26–30	Solid (**pass threshold ≥ 26**)
18–25	Gaps in causal reasoning; revisit lessons named in each question header
< 18	Re-study Phase 4 lessons before proceeding

Reasoning (40 pts). B (24) and E (16). B: 1 point for the correct result, up to 2 for justification (render-phase vs async error, import direction, ownership, one-time initial state, cascade/inheritance, ESM evaluation order, `exports` encapsulation). E: points for stated criteria, a decision consistent with them, trade-offs, and a counter-case; a "right" decision with no criteria earns at most 1 of 4; a decision that ignores migration cost or team constraints is capped at 2.

Debugging (30 pts). Each of C1–C5: 1.5 points each for cause, mechanism, fix, verification. A fix that masks the symptom (wrapping everything in a boundary and ignoring the async error path; adding `// eslint-disable`; manually reordering imports to "make the cycle work"; hoisting React into the app only on one machine) earns **0** for the fix criterion.

Implementation (30 pts). Rubrics under D1–D3. Code must type-check conceptually; syntax slips are tolerated; semantic errors (side effects in `getDerivedStateFromError`, React imports in domain code, theme state that re-renders every component, derived duplicates of server state) are scored down per rubric. Reasonable alternative designs that meet the stated requirements earn full credit if the student explains the trade-off.

Pass thresholds

- **Overall ≥ 80% (≥ 109 / 136).**
- **Each category ≥ 70%:** Explanation ≥ 26 / 36, Reasoning ≥ 28 / 40, Debugging ≥ 21 / 30, Implementation ≥ 21 / 30.

Failing any single category fails the assessment even if the total exceeds 80%. Remediate the failed category and retake **only the failed parts** with new variants (instructors: change scenarios, names and data; do not reuse the same snippets).

Remediation map

Weak area	Revisit
Architecture vocabulary, coupling/cohesion, drift (A1, E1)	01, 02
Component responsibility, presentational/container (A2, B4)	03, 04
Dependency direction, DIP, injection (A3, B7, D2)	05, 11
Feature vs layer, modules, FSD (A4, B2, C2, C5, E1)	06, 07, 09
Domain/hexagonal, application services (A5, D2)	08, 10, 12, 13
API boundaries and data access (A6, C1, D2)	14, 13
State ownership, server vs client state (A7, B4, B5, C4, E2)	15, 16
Data flow and routing (A8, E2)	17, 18
Error boundaries (A9, B1, B3, C1, D1)	19
Design systems, component libraries (A10, B8, C3, D3, E3)	20, 21
Monorepos, multi-team ownership (A11, C3, E1, E3)	22, 23
Refactoring legacy, enforcement and ratchets (A12, C2, C5, E1, E4)	24, 25
Styling architecture (A12, B6, D3, E3)	27

Promotion criteria

You may proceed to **Phase 5 — Design patterns** (Phase 5 / 01 — SOLID in React) when **all** hold:

1. You meet the pass thresholds above.
2. You can, without notes, explain: why dependency direction matters and how DIP is realised in React; the FSD/feature import rules and why same-layer slices do not import each other; the difference between ownership, location and access of state; why server data must not be copied

into local state; what error boundaries do and do not catch; why `exports`, peers and `sideEffects` determine whether a library is safe to consume; what a ratchet is and why it makes migration safe; and how tokens + CSS custom properties decouple theming from the styling tool.

3. You have completed the Phase 4 projects: **Project 3 — Multi-page dashboard** and **Project 8 — Enterprise modular app** (see `projects/`), each with a written ownership map, a dependency-rule document enforced in CI (at least one boundary rule and one cycle check), and at least one ADR explaining a structural decision and its alternatives.
4. In a 30-minute live review of an unfamiliar modular codebase you can identify at least four architectural problems from lessons 01–25 (for example cross-feature imports, copied server state, missing boundaries, leaked DTOs, cycles, shared dumping ground) and propose fixes with trade-offs and a migration path that does not stop feature work.

Conditional promotion (80–84% overall with all categories ≥ 70%): proceed, but before Phase 5 lesson 05 — Compound components and lesson 16 — Dependency injection, re-read the remediation-map lessons for your two lowest-scoring questions; Phase 5 assumes fluency with composition, injection and public-API thinking.

Re-study required: any category < 70%, or overall < 80%.

> ⚠ Items that depend on fast-moving tools (ESLint/stylelint plugin options, bundler library mode, CSS-in-JS library maintenance status, FSD specification details, RSC directive handling in bundlers) are assessed for conceptual understanding and for the habit of verifying against current documentation, not for memorised configuration.